

Security Testing and DevSecOps Automation of the Streeling University Inventory Management System

Student ID: 5653630, May 2026

The Cyber Context of Software Engineering (WM240), WMG, University of Warwick

Contents

1	Introduction	1
2	Security Testing Environment and Methodology	1
2.1	IMS System Architecture and Trust Boundaries	1
2.2	Security Testing Approach	2
2.3	Tooling Configuration	2
2.3.1	Environment Setup	2
2.3.2	Snyk Configuration	2
2.3.3	OWASP ZAP Configuration	2
3	Vulnerability Analysis and remediation	3
3.1	Snyk Scan Summary	3
3.1.1	Frontend Dependency Vulnerability	3
3.1.2	Backend Dependency Vulnerability	6
3.2	OWASP ZAP Scan Summary	7
3.2.1	Missing Content Security Policy (CSP) Header	8
3.2.2	Missing X-Frame-Options Header	10
3.2.3	Missing Subresource Integrity Attribute	12
4	Pipeline Design	14
4.1	Snyk Integration in the Pipeline	14
4.2	OWASP ZAP Integration in the Pipeline	16
4.3	Pipeline Execution Evidence	17
4.4	Evaluation of DevSecOps Automation	18
5	Conclusion	19
	References	20
	Appendices	22

A	IMS Architecture and Security Assessment Artefacts	22
B	Old Codebase	23
C	Supplementary GitHub Actions Development Evidence	24
D	DevSecOps Repository and Pipeline Artefacts - GitLab	28
E	AI Declaration and Academic Integrity	29

1 Introduction

This report presents the application of DevSecOps principles and automated security testing techniques to the previously developed Streeling University Inventory Management System (IMS). It focuses on utilising both SAST and DAST methodologies to integrate automated security testing into the software development lifecycle. The identified vulnerabilities from the assessment were then analysed and mapped to the OWASP Top Ten security risks before being evaluated through the OWASP Risk Rating methodology. In addition, a CI/CD pipeline was implemented using GitLab CI/CD to automate security testing processes and improve security visibility during the application evaluation process.

2 Security Testing Environment and Methodology

2.1 IMS System Architecture and Trust Boundaries

The existing IMS architecture was built using a modular methodology, with the final implementation utilising three logical layers to separate presentation, business logic, and data management responsibilities, each encapsulated for a specific function, as shown in the UML component diagram in Figure 1, with additional higher-level architecture and STRIDE-informed trust boundary diagrams being provided in Appendix A.

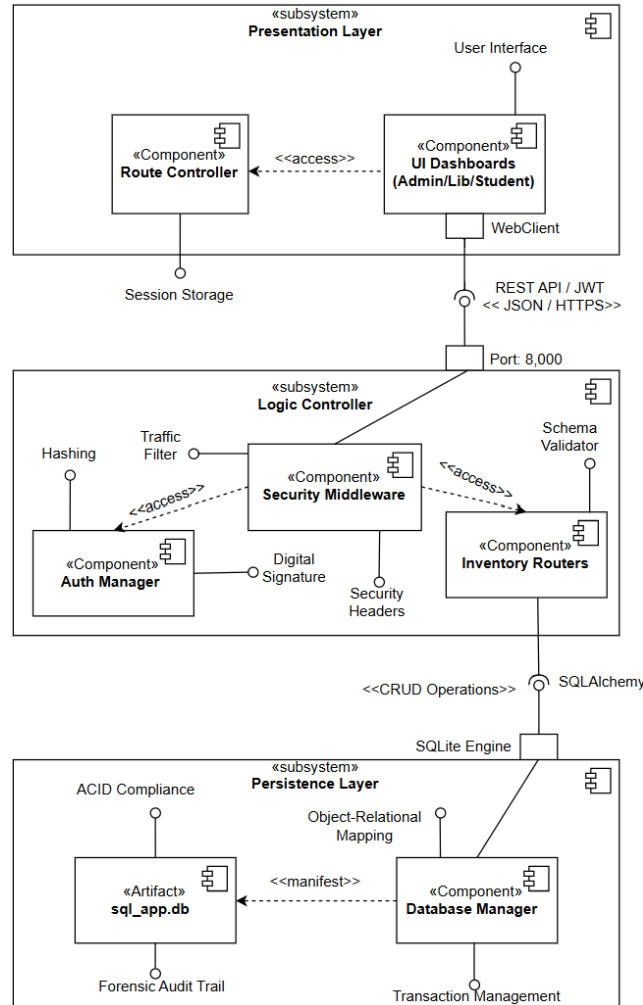


Figure 1: UML component diagram of the IMS layered architecture

2.2 Security Testing Approach

To align the IMS application with DevSecOps security practices, both Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) methodologies [1] were used throughout the assessment process. This approach ensured multiple testing methodologies were used to identify both the internal dependency risks and runtime configuration weaknesses of the application, since relying on a single testing methodology would have provided incomplete security coverage [2].

Snyk [3] was used as the primary SAST tool against the frontend and backend dependency configurations used throughout the IMS system. The scans targeted the React and FastAPI environments in order to identify vulnerable or outdated packages, known CVEs, and software supply-chain risks affecting the application components.

OWASP ZAP [4] was then used to dynamically assess the behaviour of the locally deployed application during execution. The scans focused on analysing runtime HTTP responses and browser-side security controls in order to identify security misconfigurations and missing protection mechanisms within the deployed environment. Combining both tools for their respective testing methodologies improves the overall effectiveness of the security assessment by identifying security weaknesses that would not have been detectable through static or dynamic analysis alone.

2.3 Tooling Configuration

2.3.1 Environment Setup

The IMS application was tested within a local development environment consisting of separate frontend and backend services. This is because it provides a safe place to test out the system's code, which enables developers to gain faster feedback and ship the product with confidence [5]. The frontend service was deployed locally using the React Vite development server, while the backend API was executed using FastAPI, and they were separated intentionally to consolidate APIs and make the process of switching frameworks in the future simpler [6].

2.3.2 Snyk Configuration

Static dependency analysis was performed using Snyk Open Source environments of the IMS application. The scans targeted dependency configuration files including `package.json`, `package-lock.json`, and `requirements.txt` in order to identify vulnerable dependencies, software composition risks, and publicly disclosed CVEs affecting the React frontend and FastAPI backend services, while also generating remediation guidance for affected packages.

2.3.3 OWASP ZAP Configuration

OWASP ZAP was configured to dynamically assess the locally deployed IMS application during runtime execution. The scans targeted the frontend application URL and analysed runtime HTTP behaviour, response headers, and externally observable security controls in order to identify misconfigurations and missing protection mechanisms that could not be detected through static analysis alone.

3 Vulnerability Analysis and remediation

3.1 Snyk Scan Summary

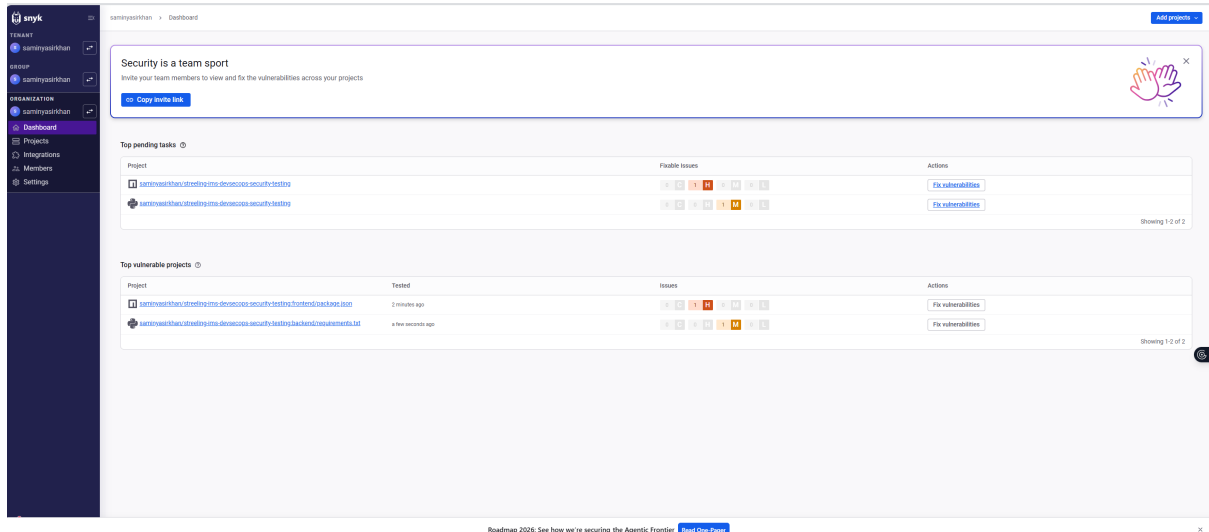


Figure 2: Snyk dashboard summary showing identified dependency vulnerabilities

3.1.1 Frontend Dependency Vulnerability

After successful registration onto the platform, the vulnerability scan was done through Snyk's GitHub repository integration feature [7], which enabled Snyk to import the entire IMS repository from GitHub and analyse dependencies (e.g. requirements.txt), in order to identify vulnerable third-party packages affecting both the frontend and backend environments.

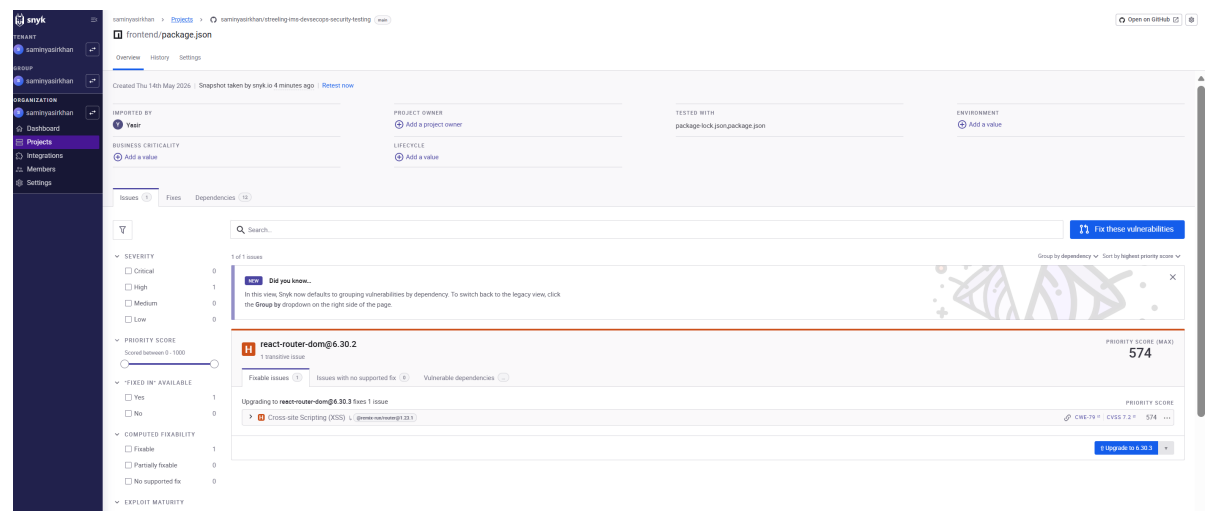


Figure 3: Snyk frontend dependency vulnerability affecting the React environment

The frontend dependency scan identified a high-severity vulnerability affecting the `react-router-dom` package used within the React environment of the IMS application, as shown in Figure 3. This is because the issue stemmed from a vulnerable transitive

dependency associated with `@remix-run/router`, which introduced a potential Cross-Site Scripting (XSS) exposure affecting browser-side application behaviour.

The affected dependency was responsible for handling client-side routing and navigation behaviour throughout the frontend application interface, which significantly weakened the security posture of web applications. This is because browser-side dependencies can increase the risk of malicious script execution and compromise user interaction within the deployed environment.

These vulnerabilities can be seen in the old codebase (Appendix B):

```
1 "react-router-dom": "^6.22.3"
```

Listing 1: Vulnerable frontend dependency in frontend/package.json (Line 17)

```
1 "node_modules/react-router-dom": {
```

Listing 2: Installed vulnerable dependency in frontend/package-lock.json (Line 4493)

```
1 "node_modules/@remix-run/router": {
```

Listing 3: Transitive vulnerable dependency identified by Snyk in frontend/package-lock.json (Line 1002)

The vulnerability was classified by Snyk as high severity with a CVSS score of 7.2 and linked to CWE-79. The platform additionally identified the appropriate recommendation needed to fix the issue, which consisted of simply upgrading the package from version 6.30.2 to 6.30.3. The vulnerability can be categorised under OWASP A06: Vulnerable and Outdated Components, since the weakness originated from the use of an outdated third-party dependency within the frontend application environment. The OWASP Risk Rating calculator additionally classified the vulnerability as relatively low risk due to the relatively high exploit likelihood associated with publicly exposed browser-side dependencies and the potential impact on user interaction security, as shown in Figure 4.

OWASP Risk Rating Calculator

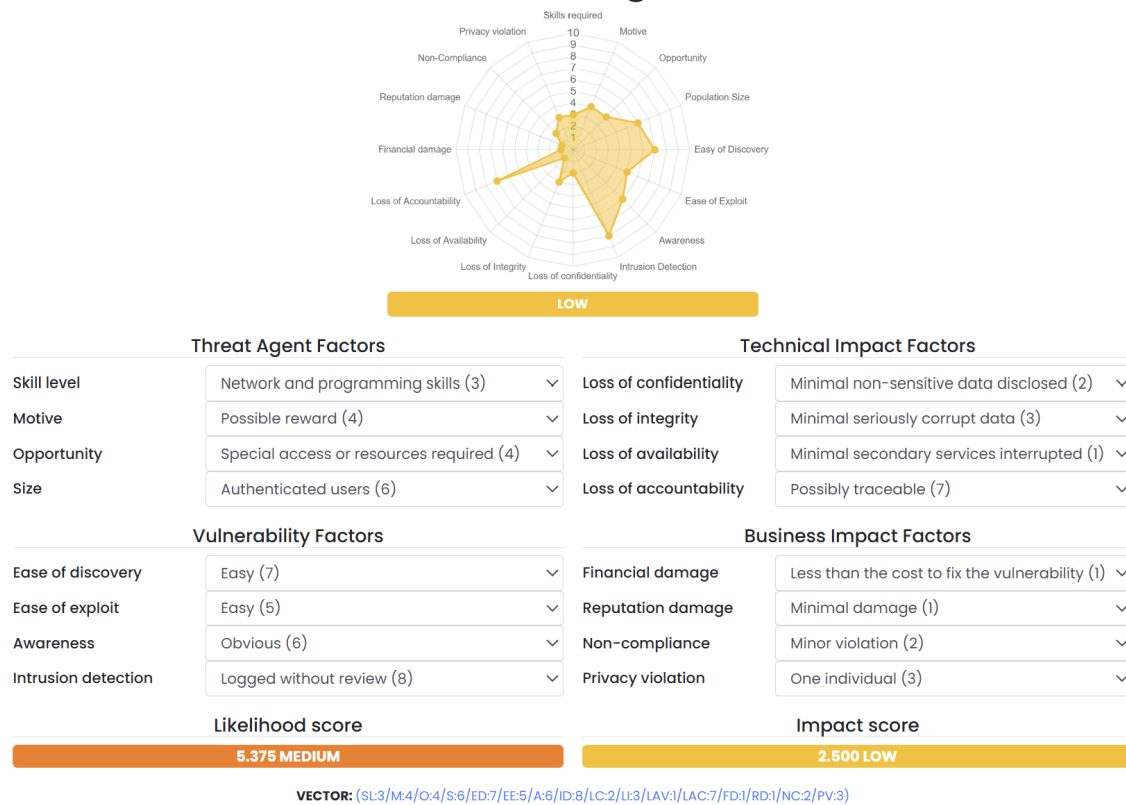


Figure 4: OWASP Risk Rating assessment for the React dependency vulnerability

As discussed above the remediation step only required a PowerShell command to upgrade the outdated dependency into a more secure version, as shown in Figure 5.

```
Active code page: 65001
C:\Users\samin\OneDrive\Warwick\Desktop\ccse-ims-coursework1-submission>c:\Users\samin\OneDrive\Warwick\Desktop\ccse-ims-coursework1-submission\.venv\Scripts\activate.bat
(.venv) C:\Users\samin\OneDrive\Warwick\Desktop\ccse-ims-coursework1-submission>npm install react-router-dom@6.30.3
added 6 packages in 4s
(.venv) C:\Users\samin\OneDrive\Warwick\Desktop\ccse-ims-coursework1-submission>
```

Figure 5: Frontend dependency upgrade remediation

3.1.2 Backend Dependency Vulnerability

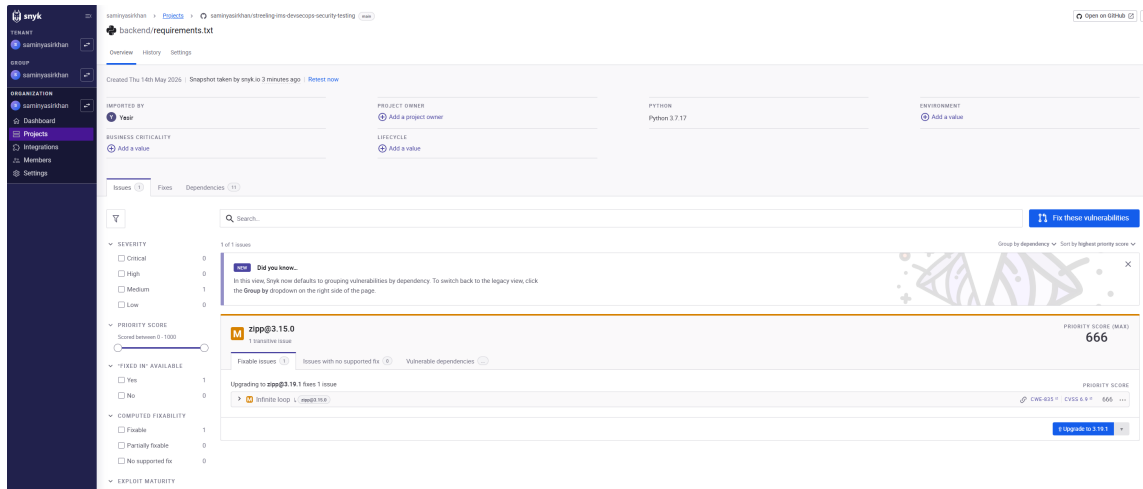


Figure 6: Snyk backend dependency vulnerability affecting the FastAPI environment

Similarly, the backend dependency analysis identified a medium-severity vulnerability affecting the Python `zipp` package version 3.15.0 inside the dependency configuration used by the FastAPI service, as shown in Figure 6. The vulnerability turned out to be associated with CWE-835, which relates to uncontrolled infinite loop conditions during software execution, and can be categorised under the OWASP A06: vulnerability, with its risk score being ranked relatively low in Figure 7.

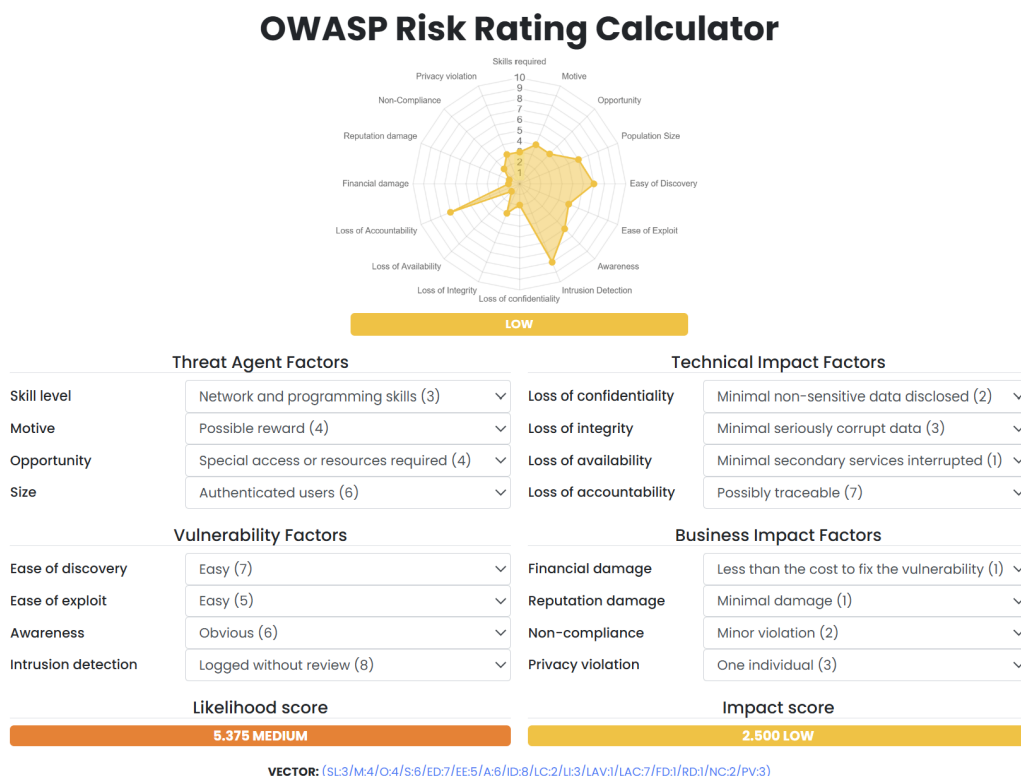


Figure 7: OWASP Risk Rating assessment for the backend dependency vulnerability

The fix therefore also required the upgrade of the `zipp` 3.19.1 dependency into a newer secure version, as displayed in Figure 8.

```
(.venv) C:\Users\samin\OneDrive\Warwick\Desktop\ccse-ims-coursework1-submission\ccse-ims-coursework1-submission>echo zipp==3.19.1>> backend\requirements.txt
(.venv) C:\Users\samin\OneDrive\Warwick\Desktop\ccse-ims-coursework1-submission\ccse-ims-coursework1-submission>type backend\requirements.txt
fastapi==0.124.0
uvicorn==0.38.0
SQLAlchemy==2.0.40
pydantic==2.12.5
bcrypt==5.0.0
python-jose[cryptography]==3.5.0
python-dotenv==1.2.1
slowapi==0.1.9
python-multipart==0.0.21
zipp==3.19.1
(.venv) C:\Users\samin\OneDrive\Warwick\Desktop\ccse-ims-coursework1-submission\ccse-ims-coursework1-submission>
```

Figure 8: Backend dependency upgrade remediation

3.2 OWASP ZAP Scan Summary

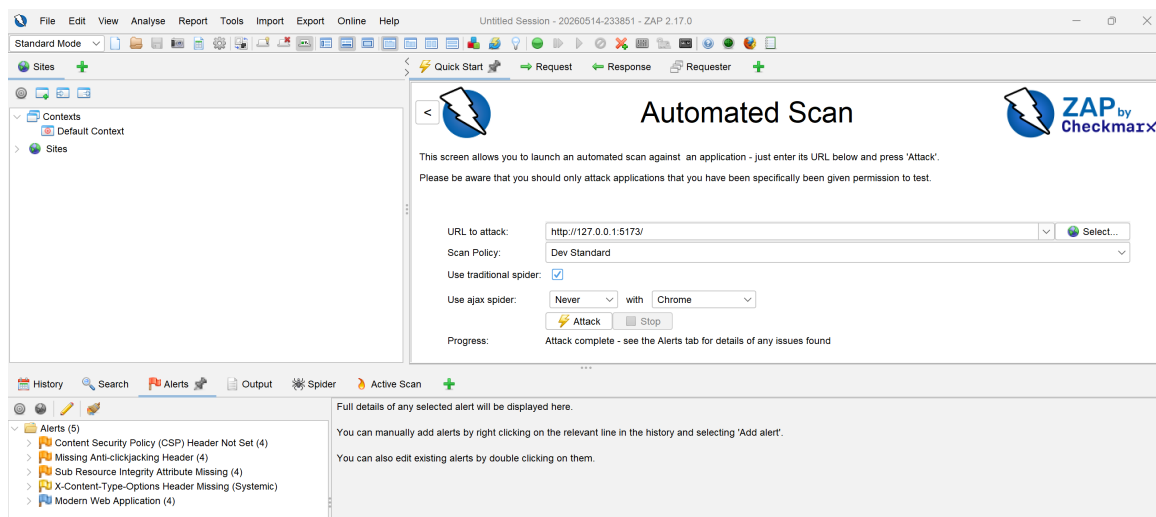


Figure 9: OWASP ZAP automated scan summary showing identified runtime vulnerabilities

The automated OWASP ZAP assessment summary, as shown in Figure 9, generated three medium-risk vulnerability alerts when scanning the running IMS environment. The scan primarily detected missing HTTP security headers and browser-side protection mechanisms, including missing Content Security Policy (CSP) configurations and anti-clickjacking protections. These findings demonstrate how DAST methodologies can be used to expose browser-based attacks during user interaction with the application.

3.2.1 Missing Content Security Policy (CSP) Header

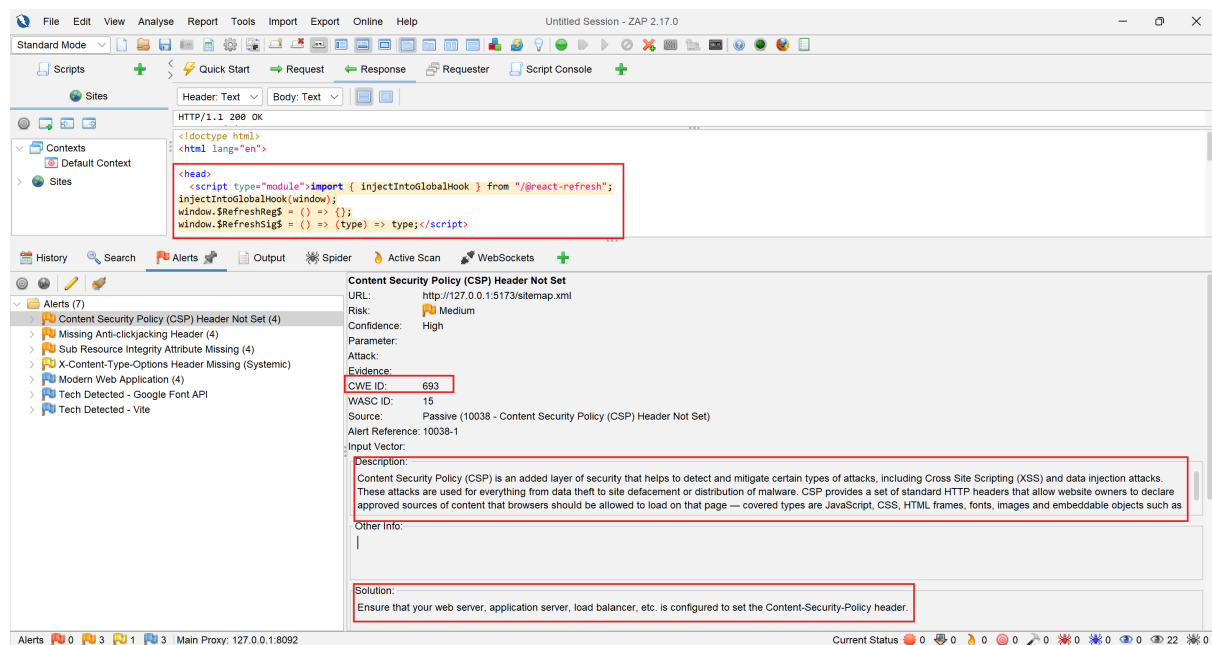


Figure 10: Scan identifying a missing CSP header

The automated DAST test shown in Figure 10 highlights the lack of Content Security Policy (CSP) within the IMS frontend framework. The finding was mapped as a medium-severity passive vulnerability associated with CWE-693, and described as a weakness where the platform does not use or incorrectly implement a protection mechanism that provides sufficient defence against targeted attacks on the IMS platform. While the vulnerability does not imply the presence of exploitability, the lack of protection on the browser side can be considered a control gap. Consequently, the absence of this control may weaken the overall security posture of the application by reducing protection against unauthorised script execution and browser-based injection attacks. The remediation improved protection for IMS dashboard interactions and reduced the risk of malicious script execution affecting JWT-based user sessions.

This can additionally be observed within the Risk Rating assessment shown in Figure 11.

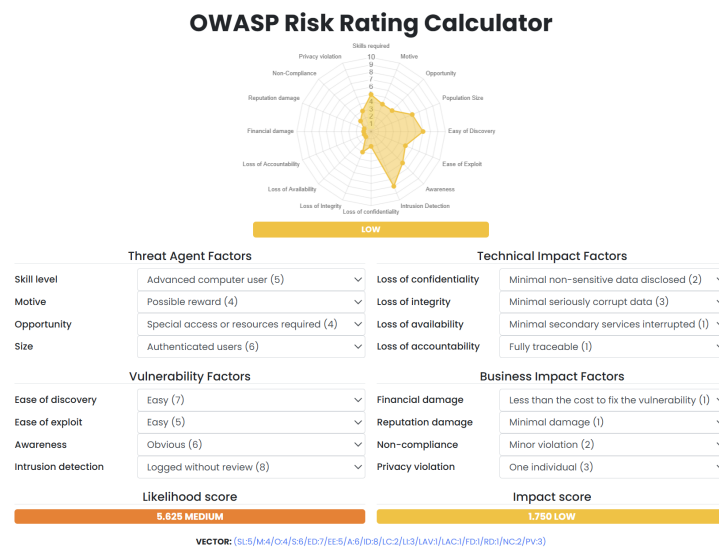


Figure 11: OWASP Risk Rating assessment for the missing CSP vulnerability

The remediation process would simply involve us adding security response headers inside the `frontend/vite.config.js`, as shown in Figure 12 and evidenced in Figure 13.

```

vite.config.js M X
frontend > vite.config.js > ...
1  import { defineConfig } from 'vite'
2  import react from '@vitejs/plugin-react'
3
4  // https://vitejs.dev/config/
5  export default defineConfig({
6    plugins: [react()],
7    server: {
8      host: '127.0.0.1',
9      port: 5173,
10     headers: {
11       'Content-Security-Policy': [
12         "default-src 'self'",
13         "script-src 'self' 'unsafe-inline'",
14         "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com",
15         "font-src 'self' https://fonts.gstatic.com data:",
16         "img-src 'self' data: https://ui-avatars.com",
17         "connect-src 'self' http://127.0.0.1:8000 ws://127.0.0.1:5173",
18         "object-src 'none'",
19         "base-uri 'self'",
20         "frame-ancestors 'none'"
21       ].join('; '),
22       'X-Frame-Options': 'DENY',
23       'X-Content-Type-Options': 'nosniff',
24       'Referrer-Policy': 'strict-origin-when-cross-origin'
25     }
26   }
27 })

```

Figure 12: CSP header configuration

```

C:\Users\lamin\OneDrive\Desktop\streeking-ins-devsecops-security-testing\frontend\curl.exe -I http://127.0.0.1:5173/
HTTP/1.1 200 OK
Vary: Origin
Content-Type: text/html
Cache-Control: no-cache
etag: W/"721-remixdevsecops-1234567890"
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com data; img-src 'self' data: https://ui-avatars.com; connect-src 'self' http://127.0.0.1:8000 ws://127.0.0.1:5173; object-src 'n
one'; base-uri 'self'; frame-ancestors 'none'
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Date: Sun, 17 May 2020 18:28:27 GMT
Connection: keep-alive
Keep-Alive: timeout=5

C:\Users\lamin\OneDrive\Desktop\streeking-ins-devsecops-security-testing\frontend\cd ..
C:\Users\lamin\OneDrive\Desktop\streeking-ins-devsecops-security-testing\backend
C:\Users\lamin\OneDrive\Desktop\streeking-ins-devsecops-security-testing\backend\curl.exe -I http://127.0.0.1:8000/
HTTP/1.1 405 Method Not Allowed
date: Sun, 17 May 2020 18:28:41 GMT
server: uicorn
allow: GET
content-length: 31
content-type: application/json
x-content-type-options: nosniff
x-frame-options: DENY
x-protection: 1; mode=block
referrer-policy: strict-origin-when-cross-origin
content-security-policy: default-src 'self'; script-src 'self' data: https://ui-avatars.com; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline';
permissions-policy: camera(), microphone(), geolocation(), payment()

C:\Users\lamin\OneDrive\Desktop\streeking-ins-devsecops-security-testing\backend\curl.exe http://127.0.0.1:8000/
{"message": "IMS Backend Service - Modularized!"}
C:\Users\lamin\OneDrive\Desktop\streeking-ins-devsecops-security-testing\backend

```

Figure 13: Runtime header validation

3.2.2 Missing X-Frame-Options Header

The screenshot shows the ZAP interface with the following details:

- Alerts (7):**
 - Content Security Policy (CSP) Header Not Set (4)
 - Missing Anti-clickjacking Header (4)
 - Sub Resource Integrity Attribute Missing (4)
 - X-Content-Type-Options Header Missing (Systemic)
 - Modern Web Application (4)
 - Tech Detected - Google Font API
 - Tech Detected - Vite
- Missing Anti-clickjacking Header**
 - URL: http://127.0.0.1:5173/sitemap.xml
 - Risk: Medium
 - Confidence: Medium
 - Parameter: x-frame-options
 - Attack:
 - Evidence:
 - CWE ID: 1021
 - WASC ID: 15
 - Source: Passive (10020 - Anti-clickjacking Header)
 - Alert Reference: 10020-1
 - Input Vector:
 - Description: The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
 - Other Info:
 - Solution: Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be

Figure 14: Scan identifying a missing X-Frame-Options header

Figure 14 further displays results from automatic scans, which demonstrated additional vulnerability identifications, such as the medium-risk CWE-1021 vulnerability, which does not restrict or incorrectly restrict frame objects or UI layers that belong to another application or domain. This highlights additional browser-side security control weaknesses identified during the automated DAST assessment.

The likelihood and impact of the vulnerability can additionally be observed in Figure 15.

OWASP Risk Rating Calculator



Figure 15: OWASP Risk Rating assessment for missing X-Frame-Options vulnerability

As evidenced in Figure 16 and Remediated in Figure 12, the solution involved adding X-Frame-Options: DENY and CSP frame-ancestors 'none' headers to the Vite front-end server responses.

```
PS C:\Users\samin\OneDrive\Warwick\Desktop\streeling-ims-devsecops-security-testing> curl.exe -I http://127.0.0.1:5173/sitemap.xml
HTTP/1.1 200 OK
Vary: Origin
Content-Type: text/html
Cache-Control: no-cache
Etag: W/"371-rxRYsHaXKPLtyvICUBYR/fQpA9Q"
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com data; img-src 'self' data: https://ui-avatars.com; connect-src 'self' http://127.0.0.1:8000 ws://127.0.0.1:5173; object-src 'none'; base-uri 'self'; frame-ancestors 'none'
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Date: Sun, 17 May 2026 19:14:14 GMT
Connection: keep-alive
Keep-Alive: timeout=5

PS C:\Users\samin\OneDrive\Warwick\Desktop\streeling-ims-devsecops-security-testing> |
```

Figure 16: Runtime header validation 2

3.2.3 Missing Subresource Integrity Attribute

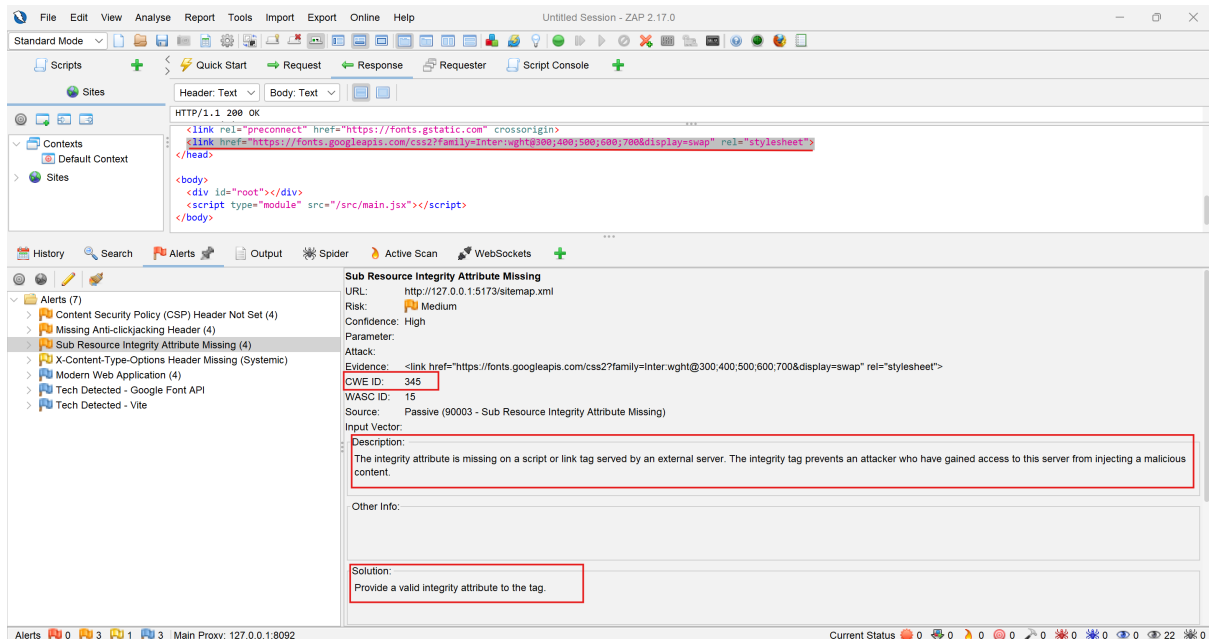


Figure 17: Scan identifying a missing Subresource Integrity (SRI) attribute vulnerability

The runtime vulnerability assessment presented in Figure 17 identified a missing Subresource Integrity (SRI) attribute within externally loaded frontend resources. The vulnerability was mapped to the category CWE-345 and classified as a medium-risk browser-side vulnerability. The vulnerability exists because externally hosted resources, such as Google Fonts stylesheets, were loaded without integrity verification, which may increase the risk of malicious content injection if the third-party resource becomes compromised.

The vulnerability additionally demonstrates weaknesses in frontend supply-chain trust validation, since browsers are unable to verify whether externally retrieved resources have been modified before execution within the client environment. Consequently, the absence of SRI protection may weaken the integrity assurance of externally loaded frontend assets. This can additionally be observed within the Risk Rating assessment shown in Figure 18.

OWASP Risk Rating Calculator

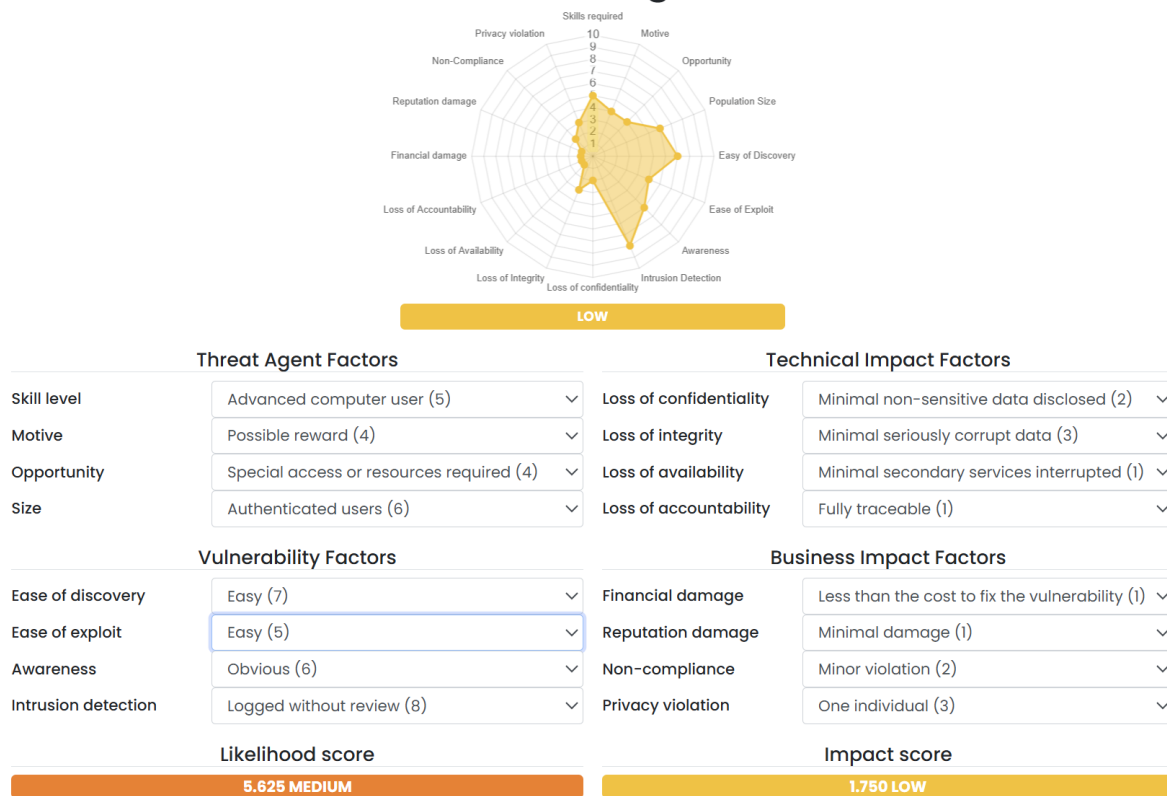


Figure 18: OWASP Risk Rating assessment for missing SRI vulnerability

The remediation process involved removing the externally hosted Google Fonts stylesheet from the frontend HTML and replacing the Inter font dependency with a local system font stack. In addition, the Content Security Policy was tightened so external Google Fonts domains were no longer permitted sources.

```
PS C:\Users\samin> curl.exe http://127.0.0.1:5173/ | Select-String "fonts.googleapis|fonts.gstatic|stylesheet"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100    600  100    600    0      0    2041    0
PS C:\Users\samin>
```

Figure 19: Runtime validation showing removal of externally hosted Google Fonts stylesheet references

4 Pipeline Design

GitLab CI/CD was used to implement and evidence the DevSecOps pipeline prior to final deployment and repository hosting (Appendix D), since GitLab provides integrated CI/CD automation for building, testing, and validating projects directly from the submitted repository [8].

4.1 Snyc Integration in the Pipeline

```
name: DevSecOps Security Pipeline

on:
  push:
  pull_request:

permissions:
  contents: read

jobs:
  snyk-sast:
    name: Snyc SAST and Dependency Scan
    runs-on: ubuntu-latest

    env:
      SNYK_TOKEN: ${ secrets.SNYK_TOKEN }

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install backend dependencies
        working-directory: backend
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm
          cache-dependency-path: frontend/package-lock.json

      - name: Install frontend dependencies
        working-directory: frontend
        run: npm install

      - name: Install Snyc CLI
        uses: snyk/actions/setup@master

      - name: Verify Snyc authentication
        run: |
          if [ -z "${SNYK_TOKEN}" ]; then
            echo "::error::SNYK_TOKEN is not configured. Add it in GitHub under Settings > Secrets and variab
            exit 1
          fi

          snyk --version
          snyk auth "${SNYK_TOKEN}"

      - name: Create security report directory
        run: mkdir -p security-reports
```

Figure 20: Snyc CI/CD setup and secure token authentication


```

- name: Start backend server
  working-directory: backend
  run: |
    echo "Starting FastAPI backend on http://127.0.0.1:8000 ..."
    nohup python -m uvicorn main:app --host 127.0.0.1 --port 8000 > ../security-reports/backend-runtime.log 2> &

    for attempt in {1..30}; do
      if curl --fail --silent http://127.0.0.1:8000/ > /dev/null; then
        echo "Backend is ready."
        exit 0
      fi
      echo "Waiting for backend... attempt ${attempt}"
      sleep 2
    done

    echo "::error::Backend did not become ready for DAST scanning."
    cat ../security-reports/backend-runtime.log
    exit 1

- name: Start frontend server
  working-directory: frontend
  run: |
    echo "Starting Vite frontend on http://127.0.0.1:5173 ..."
    nohup npm run dev -- --host 127.0.0.1 --port 5173 > ../security-reports/frontend-runtime.log 2>&1 &

    for attempt in {1..30}; do
      if curl --fail --silent http://127.0.0.1:5173/ > /dev/null; then
        echo "Frontend is ready."
        exit 0
      fi
      echo "Waiting for frontend... attempt ${attempt}"
      sleep 2
    done

    echo "::error::Frontend did not become ready for DAST scanning."
    cat ../security-reports/frontend-runtime.log
    exit 1

- name: OWASP ZAP baseline scan
  uses: zaproxy/action-baseline@v0.15.0
  with:
    target: "http://127.0.0.1:5173"
    docker_name: "ghcr.io/zaproxy/zaproxy:stable"
    allow_issue_writing: false
    fail_action: false
    artifact_name: zap-baseline-reports
    cmd_options: "-r report_html.html -J report_json.json -w report_md.md"

- name: Upload application runtime logs

```

Figure 21: Runtime startup and OWASP ZAP baseline scan configuration

Figure 20 and Figure 21 illustrate the initial CI/CD security configuration used to design the DevSecOps pipeline before final GitLab CI/CD validation. The pipeline was set up to run automatically on repository commits in order to ensure continuous security testing during development.

Snyk was employed as an automatic SAST and dependency scanning tool before runtime deployment. Three scans were set up separately for backend and frontend codebases because each uses a different dependency, namely Python/pip and Node/npm. Authentication was securely handled using the `SNYK_TOKEN` GitLab CI/CD variable, as opposed to hard-coding credentials into the `.gitlab-ci.yml` file.

The pipeline generated structured JSON vulnerability reports and uploaded them as GitLab job artifacts for auditability and evidence retention. In addition, a high-severity security gate was introduced in order to intentionally block insecure builds whenever critical vulnerabilities were detected.

```

2593 21:26:00 $ mkdir -p security-reports
2594 21:26:00 $ test -n "$SNYK_TOKEN"
2595 21:26:00 $ snyk --version
2596 21:26:02 1.1305.0
2597 21:26:03 $ snyk auth "$SNYK_TOKEN"
2598 21:26:06 Your account has been authenticated. Snyk is now ready to be used.

```

Figure 22: Snyk authentication in the GitLab CI/CD job

Following the workflow configuration, the runtime execution evidence presented in Figure 22 confirms that the GitLab CI/CD job executed successfully after the Snyk integration was configured. The pipeline was configured to record Snyk scan results and preserve them as generated artefacts, demonstrating the security gate for DevSecOps was functioning as expected. Moreover, security reports were automatically generated and published as GitLab job artifacts to support auditing and remediation activities.

4.2 OWASP ZAP Integration in the Pipeline

```

2784 21:27:01 Backend is ready.
2785 21:27:01 $ for attempt in $(seq 1 30); do # collapsed multi-line command
2786 21:27:04 Frontend is ready.
2787 21:27:04 $ cd /zap/wrk
2788 21:27:04 $ zap-baseline.py -t "$ZAP_TARGET" -r report_html.html -J report_json.json -w report_md.md || true
2789 21:27:50 Using the Automation Framework
2790 21:27:50 Total of 8 URLs
2791 21:27:50 PASS: Vulnerable JS Library (Powered by Retire.js) [10003]
2792 21:27:50 PASS: In Page Banner Information Leak [10009]
2793 21:27:50 PASS: Cookie No HttpOnly Flag [10010]
2794 21:27:50 PASS: Cookie Without Secure Flag [10011]
2795 21:27:50 PASS: Re-examine Cache-control Directives [10015]
2796 21:27:50 PASS: Cross-Domain JavaScript Source File Inclusion [10017]
2797 21:27:50 PASS: Content-Type Header Missing [10019]
2798 21:27:50 PASS: Anti-clickjacking Header [10020]
2799 21:27:50 PASS: X-Content-Type-Options Header Missing [10021]
2800 21:27:50 PASS: Information Disclosure - Debug Error Messages [10023]
2801 21:27:50 PASS: Information Disclosure - Sensitive Information in URL [10024]
2802 21:27:50 PASS: Information Disclosure - Sensitive Information in HTTP Referrer Header [10025]
2803 21:27:50 PASS: HTTP Parameter Override [10026]
2804 21:27:50 PASS: Information Disclosure - Suspicious Comments [10027]
2805 21:27:50 PASS: Off-site Redirect [10028]
2806 21:27:50 PASS: Cookie Poisoning [10029]
2807 21:27:50 PASS: User Controllable Charset [10030]
2808 21:27:50 PASS: User Controllable HTML Element Attribute (Potential XSS) [10031]
2809 21:27:50 PASS: Viewstate [10032]
2810 21:27:50 PASS: Directory Browsing [10033]
2811 21:27:50 PASS: Heartbleed OpenSSL Vulnerability (Indicative) [10034]
2812 21:27:50 PASS: Strict-Transport-Security Header [10035]
2813 21:27:50 PASS: HTTP Server Response Header [10036]
2814 21:27:50 PASS: Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s) [10037]
2815 21:27:50 PASS: Content Security Policy (CSP) Header Not Set [10038]
2816 21:27:50 PASS: X-Backend-Server Header Information Leak [10039]
2817 21:27:50 PASS: Secure Pages Include Mixed Content [10040]
2818 21:27:50 PASS: HTTP to HTTPS Insecure Transition in Form Post [10041]
2819 21:27:50 PASS: HTTPS to HTTP Insecure Transition in Form Post [10042]
2820 21:27:50 PASS: User Controllable JavaScript Event (XSS) [10043]
2821 21:27:50 PASS: Big Redirect Detected (Potential Sensitive Information Leak) [10044]
2822 21:27:50 PASS: Retrieved from Cache [10050]
2823 21:27:50 PASS: X-ChromeLogger-Data (XCOLD) Header Information Leak [10052]
2824 21:27:50 PASS: Cookie without SameSite Attribute [10054]
2825 21:27:50 PASS: X-Debug-Token Information Leak [10056]
2826 21:27:50 PASS: Username Hash Found [10057]
2827 21:27:50 PASS: X-AspNet-Version Response Header [10061]
2828 21:27:50 PASS: PII Disclosure [10062]
2829 21:27:50 PASS: Timestamp Disclosure [10096]
2830 21:27:50 PASS: Hash Disclosure [10097]
2831 21:27:50 PASS: Cross-Domain Misconfiguration [10098]
2832 21:27:50 PASS: Source Code Disclosure [10099]
2833 21:27:50 PASS: Weak Authentication Method [10105]
2834 21:27:50 PASS: Reverse Tabnabbing [10108]
2835 21:27:50 PASS: Dangerous JS Functions [10110]

```

Figure 23: OWASP ZAP baseline scan execution against the running IMS frontend

The runtime execution evidence shown in Figure 23 presents the successful integration of OWASP ZAP within the GitLab CI/CD pipeline. Unlike SAST tools, which analyse source code and dependencies statically, OWASP ZAP required the application to be running during testing. This was achieved by deploying the front-end and back-end services in the GitLab runner environment before running the ZAP baseline test.

The pipeline additionally generated automated security artifacts, including ZAP baseline reports and application runtime logs. The use of these artifacts by uploading them as GitLab job artifacts increased the traceability and ensured that the vulnerability results could be kept for future analysis. After the final GitLab CI/CD configuration was corrected, the runtime results showed successful execution of the automated DAST phase and report generation.

The inclusion of these security measures in the CI/CD pipeline not only helped make the security assessment process more repeatable and consistent but also ensured that any vulnerabilities could be detected continually throughout the software development lifecycle, and not just at manual review stages [9]. Furthermore, this also strengthened the overall auditability of the IMS environment, since generated security artifacts and automated scan outputs could be retained as GitLab job artifacts for future analysis and remediation tracking.

4.3 Pipeline Execution Evidence

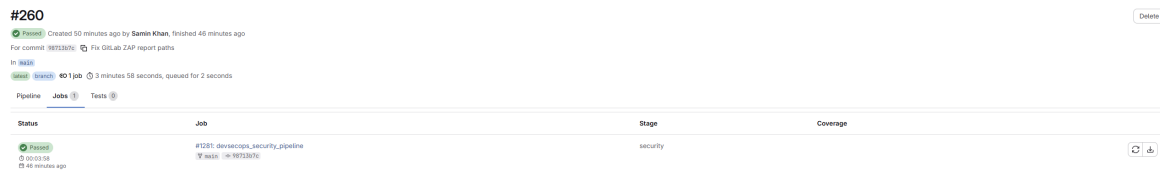


Figure 24: Successful GitLab CI/CD pipeline execution on the main branch

The GitLab CI/CD execution evidence shown in Figure 24 demonstrates the successful integration of automated DAST assessment within the CI/CD pipeline. The workflow produced several security artifacts such as OWASP ZAP baseline reports, Snyk security reports, and application runtime log files, enhancing traceability and visibility during the whole assessment process. In addition, generating these reports automatically during CI/CD execution improved the consistency of runtime security testing and reduced reliance on manual assessment procedures.

The retention of automated security artifacts within GitLab CI/CD was also consistent with the requirement for accountability and evidence preservation, as found in the UK GDPR and ISO/IEC 27001 frameworks, which require that organisations have mechanisms for monitoring security and tracking incidents [10].

4.4 Evaluation of DevSecOps Automation

The implementation of DevSecOps automation throughout the IMS assessment significantly improved the platform's overall security posture, by continuously combining SAST and DAST methodologies directly into the CI/CD pipeline. This is because Snyk focused on dependency and software composition risks, while OWASP ZAP provided runtime analysis of HTTP responses and browser security controls, enabling broader visibility across the deployed application environment. This approach not only supported secure-by-default deployment practices, but also reduced reliance on manual documentation, which improved the software's auditability within the development lifecycle. Within the IMS environment specifically, these remediation measures strengthened browser-side protections affecting JWT-based session interactions, frontend routing behaviour, and externally loaded resources. Consequently, common browser-based attacks such as clickjacking, malicious script execution, and supply-chain manipulation risks were reduced across the deployed student inventory interface.

Although the DevSecOps implementation successfully improved continuous vulnerability visibility, several limitations still remain within the implemented DevSecOps pipeline. For instance, the OWASP ZAP baseline scan focused primarily on passive browser-side assessment and therefore did not run more advanced authenticated tests to detect any potential issues with the functionality of the application. As a result, vulnerabilities affecting internal business logic, application workflows, or role-based access controls may still remain undetected within the deployed IMS environment.

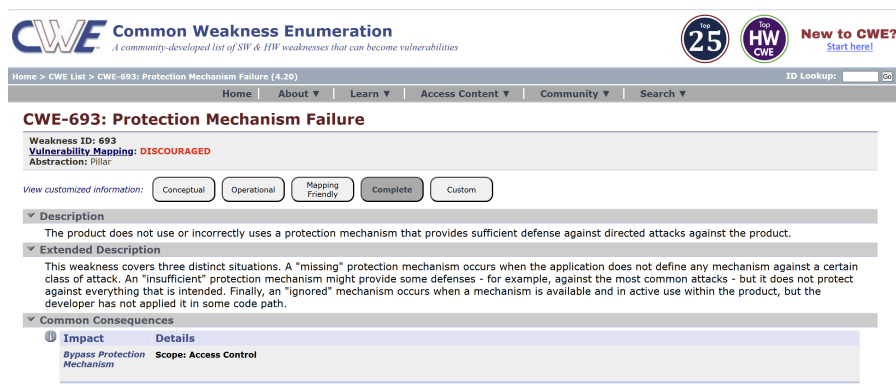


Figure 25: CWE-693 classification marked as discouraged for direct vulnerability mapping

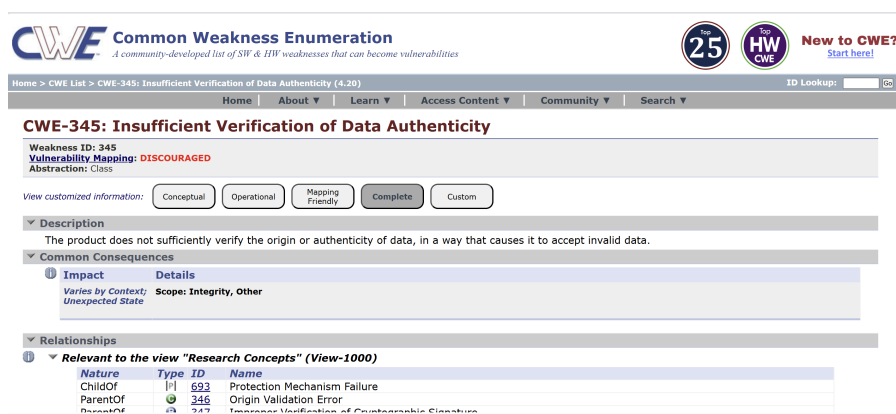


Figure 26: CWE-345 classification marked as discouraged for direct vulnerability mapping

As illustrated in Figures 25 and 26, several findings that were generated using this automated assessment process, and later analysed in the report, were both discouraged mappings for direct vulnerability identification inside the CWE database. This is because these tools rely on broad CWE classifications and may therefore generate overly generic vulnerability mappings that require additional manual analysis before the practical impact of the findings can be accurately evaluated [11].

Potential enhancements to the implementation of the DevSecOps could further strengthen the maturity of the automated security assessment environment. For example, the use of authenticated OWASP ZAP scans could enhance the assessment process by analysing application features and role-based access control. Additional security tooling such as Infrastructure-as-Code (IaC) [12] analysis, container vulnerability scanning, and automated secret detection could also improve security visibility across deployment environments. However, even though the developed workflow is capable of conducting an effective automated vulnerability assessment process, human intervention and contextual risk analysis remain crucial elements of secure software governance.

5 Conclusion

This report successfully evidences the application of DevSecOps principles to facilitate the detection and analysis of SAST and DAST scan vulnerabilities, followed by the implementation of secure remediation strategies aimed to integrate CI/CD automated security testing and improve the overall maturity of the software development lifecycle.

References

- [1] Oluwaferanmi Aremu and Warren Liang. “Static and Dynamic Application Security Testing (SAST and DAST) Integration for Robust Secure Coding Practices”. In: (July 2025).
- [2] Lyubka Dencheva. “Comparative Analysis of Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) by Using Open-Source Web Application Penetration Testing Tools”. MSc Thesis. National College of Ireland, Aug. 2022.
- [3] Snyk. *What’s Snyk?* Accessed: 2026-05-15. 2026. URL: <https://docs.snyk.io/discover-snyk/whats-snyk>.
- [4] Pritesh K. *OWASP ZAP Tutorial: A Comprehensive Guide for DevSecOps*. Accessed: 2026-05-15. May 2025. URL: <https://devsecopsschool.com/blog/owasp-zap-tutorial-a-comprehensive-guide-for-devsecops/>.
- [5] Shamal Jayawardhana. *How to Test Your Web App Locally: 3 Beginner-Friendly Ways Using a Local HTTP Server*. Accessed: 2026-05-15. July 2025. URL: https://dev.to/shamal_jayawardhana_9846c/how-to-test-your-web-app-locally-3-beginner-friendly-ways-using-a-local-http-server-3ip3.
- [6] Vikram Joshi. *Seven Reasons Why A Website’s Front-End And Back-End Should Be Kept Separate*. Forbes Technology Council, Accessed: 2026-05-15. July 2018. URL: <https://www.forbes.com/sites/forbestechcouncil/2018/07/19/seven-reasons-why-a-websites-front-end-and-back-end-should-be-kept-separate/>.
- [7] Snyk. *Snyk for GitHub User Documentation*. Accessed: 2026-05-16. 2026. URL: <https://docs.snyk.io/scm-integrations/github>.
- [8] Jacob Schmitt. *GitLab vs. GitHub: Choosing the Right Version Control Service*. CircleCI Blog, Senior Technical Content Marketing Manager, Accessed: 2026-05-18. 2024. URL: <https://circleci.com/blog/gitlab-vs-github/>.
- [9] Adedamola Abiodun Solanke. “Enterprise DevSecOps: Integrating Security into CI/CD Pipelines for Regulated Industries”. In: *World Journal of Advanced Research and Reviews* 13.2 (Feb. 2022). Review Article, Accessed: 2026-05-19, pp. 633–648. URL: <https://wjarr.com/content/enterprise-devsecops-integrating-security-cicd-pipelines-regulated-industries>.
- [10] John Whiting. *ISO 27001:2022 Annex A 8.15 – Logging*. ISMS.online, Accessed: 2026-05-19. Sept. 2025. URL: <https://www.isms.online/iso-27001/annex-a-2022/8-15-logging-2022/>.
- [11] Kaixuan Li et al. “Comparison and Evaluation on Static Application Security Testing (SAST) Tools for Java”. In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE ’23)*. Available at Singapore Management University Institutional Knowledge Repository. San Francisco, USA, Dec. 2023, pp. 921–933. URL: https://ink.library.smu.edu.sg/cgi/viewcontent.cgi?article=9979&context=sis_research.

- [12] Sritej Panchumarthi and Divya Sakamuri. *Embedding Security into Pipelines: A DevSecOps Blueprint*. TechRxiv preprint, Version 1, Accessed: 2026-05-21. June 2025. URL: <https://www.techrxiv.org/doi/full/10.36227/techrxiv.175022275.50175032/v1>.

Appendices

A IMS Architecture and Security Assessment Artefacts

The structure perspectives represented in Figure 1 can be illustrated further by Figure 27, which displays the higher-level communication flows, external services, and technology interactions used throughout the deployed environment.

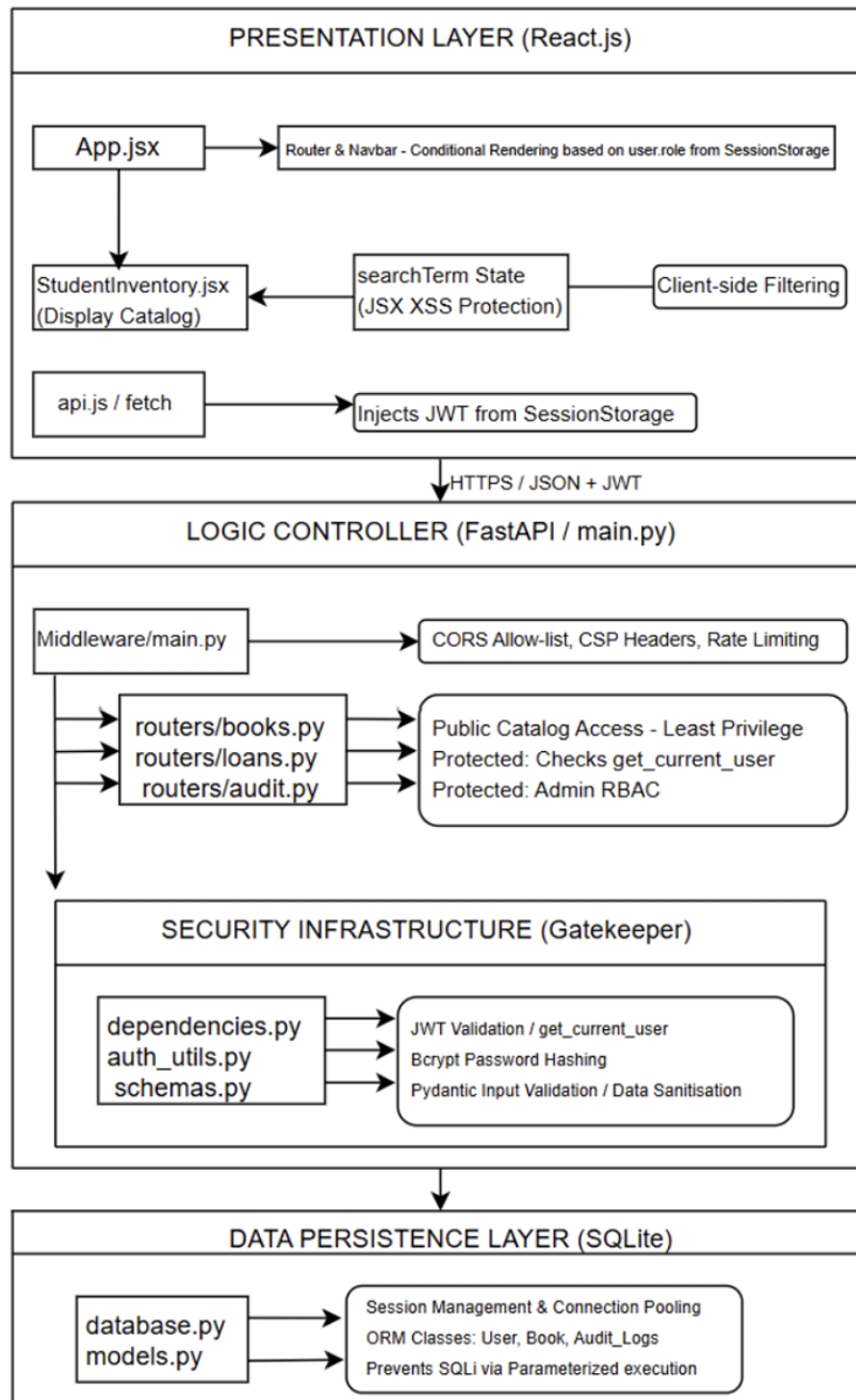


Figure 27: High-level IMS architecture and technology interactions

These illustrations of the internal implementation and high-level architecture of the IMS application can then be complemented by a STRIDE-informed threat modelling diagram, as shown in Figure 28. This diagram presents the trust boundary that exists between the users and the University IMS, by only enabling the users the operational rights required to complete their tasks, which ensures least privilege is maintained throughout the application.

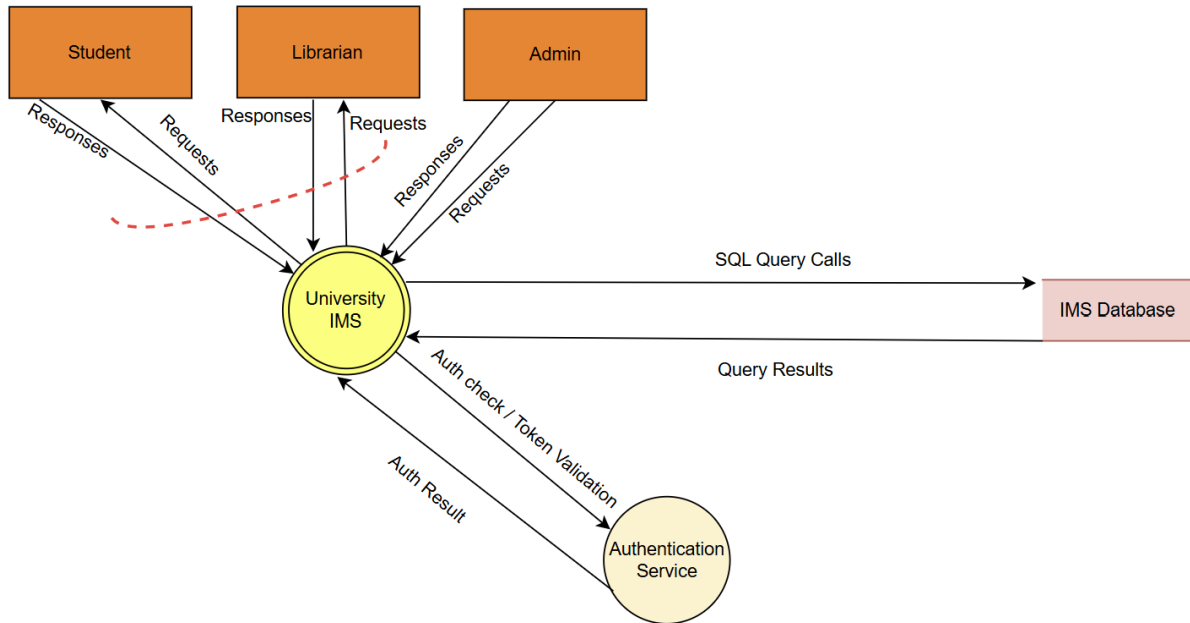


Figure 28: IMS trust boundaries and external communication flows

B Old Codebase

The complete source code for the old submission of the Inventory Management System (IMS).

Repository Access

- **Repository URL:**
https://git-lab.cyber.warwick.ac.uk/c5653630/ccse-ims-coursework1/-/tree/submission?ref_type=heads
- **Access Type:** Private (University-authenticated access)
- **Branch Used for Submission:** submission

C Supplementary GitHub Actions Development Evidence

The complete remediated source code, CI/CD workflow configuration, setup instructions, and DevSecOps pipeline artefacts for the WM240 security testing submission are available in the repository below.

Repository Access

- **Repository URL:**
`https://github.com/saminyasirkhan/streeling-ims-devsecops-security-testing`
- **Access Type:** Public GitHub repository
- **Branch Used for Submission:** main
- **Workflow File:**
`.github/workflows/devsecops-pipeline.yml`
- **Workflow Name:**
DevSecOps Security Pipeline

Repository Contents

- **Backend source code:** backend/
- **Frontend source code:** frontend/
- **Backend dependencies:** backend/requirements.txt
- **Frontend dependencies:** frontend/package.json and frontend/package-lock.json
- **Setup and verification guidance:** README.md
- **CI/CD security workflow:** .github/workflows/devsecops-pipeline.yml

Pipeline Artefacts

The GitHub Actions workflow generates equivalent DevSecOps security evidence during execution, including Snyk reports, OWASP ZAP baseline scan reports, and application runtime logs, which were mapped to the final GitLab CI/CD implementation presented throughout Section 4.

- **Snyk report artifact:** snyk-security-reports
- **OWASP ZAP report artifact:** zap-baseline-reports
- **Runtime log artifact:** application-runtime-logs

Submission Note

The complete remediated source code, CI/CD pipeline configuration, setup instructions, and DevSecOps pipeline artefacts for the WM240 security testing submission were maintained across both GitLab CI and GitHub Actions environments during the implementation and validation stages of the DevSecOps workflow.

GitHub Actions was initially used to implement and validate the DevSecOps CI/CD workflow prior to final GitLab CI/CD deployment and repository hosting, enabling automated security testing, runtime validation, and pipeline artefact generation throughout the software development lifecycle.

Supplementary GitHub Actions Pipeline Evidence

```
env:
  SNYK_TOKEN: ${ secrets.SNYK_TOKEN }

steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Set up Python
    uses: actions/setup-python@v5
    with:
      python-version: "3.11"

  - name: Install backend dependencies
    working-directory: backend
    run: |
      python -m pip install --upgrade pip
      pip install -r requirements.txt

  - name: Set up Node.js
    uses: actions/setup-node@v4
    with:
      node-version: "20"
      cache: npm
      cache-dependency-path: frontend/package-lock.json

  - name: Install frontend dependencies
    working-directory: frontend
    run: npm install

  - name: Install Snyk CLI
    uses: snyk/actions/setup@master

  - name: Verify Snyk authentication
    run: |
      if [ -z "${SNYK_TOKEN}" ]; then
        echo "::error::SNYK_TOKEN is not configured. Add it in GitHub under Settings > Secrets and variables > Actions."
        exit 1
      fi

      snyk --version
      snyk auth "${SNYK_TOKEN}"

  - name: Create security report directory
    run: mkdir -p security-reports

  - name: Snyk scan backend dependencies
    working-directory: backend
    run: |
      set +e
      echo "Running Snyk high-severity scan for backend Python dependencies..."
      snyk test \
        --file=requirements.txt \
        --package-manager=pip \
        --severity-threshold=high \
        --json-file-output=../security-reports/snyk-backend.json
      echo $? > ../security-reports/snyk-backend.exitcode
      exit 0
```

Figure 29: Secure Snyk authentication, backend/frontend dependency installation, and automated SAST dependency scanning integration within the DevSecOps CI/CD pipeline

```

run: |
  echo "Starting Vite frontend on http://127.0.0.1:5173 ..."
  nohup npm run dev -- --host 127.0.0.1 --port 5173 > ../security-reports/frontend-runtime.log 2>&1 &

  for attempt in {1..30}; do
    if curl --fail --silent http://127.0.0.1:5173/ > /dev/null; then
      echo "Frontend is ready."
      exit 0
    fi
    echo "Waiting for frontend... attempt ${attempt}"
    sleep 2
  done

  echo "::error::Frontend did not become ready for DAST scanning."
  cat ../security-reports/frontend-runtime.log
  exit 1

- name: OWASP ZAP baseline scan
  uses: zaproxy/action-baseline@v0.15.0
  with:
    target: "http://127.0.0.1:5173"
    docker_name: "ghcr.io/zaproxy/zaproxy:stable"
    allow_issue_writing: false
    fail_action: false
    artifact_name: zap-baseline-reports
    cmd_options: "-r report_html.html -J report_json.json"

- name: Upload application runtime logs
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: application-runtime-logs
    path: |
      security-reports/backend-runtime.log
      security-reports/frontend-runtime.log
    if-no-files-found: warn

- name: Enforce Snyk high-severity gate
  if: always()
  run: |
    BACKEND_EXIT=$(cat security-reports/snyk-backend.exitcode)
    FRONTEND_EXIT=$(cat security-reports/snyk-frontend.exitcode)

    echo "Backend Snyk exit code: ${BACKEND_EXIT}"
    echo "Frontend Snyk exit code: ${FRONTEND_EXIT}"

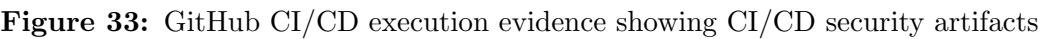
    if [ "${BACKEND_EXIT}" -eq 1 ] || [ "${FRONTEND_EXIT}" -eq 1 ]; then
      echo "::error::Snyk found high-severity vulnerabilities. The DevSecOps gate blocked the pipeline."
      exit 1
    fi

    if [ "${BACKEND_EXIT}" -ne 0 ] || [ "${FRONTEND_EXIT}" -ne 0 ]; then
      echo "::error::Snyk scan did not complete successfully. Check the SNYK_TOKEN secret and scan logs."
      exit 1
    fi

    echo "Snyk high-severity gate passed."

```

Figure 30: OWASP ZAP baseline DAST integration, runtime log artifact generation, and automated high-severity Snyk security gate enforcement



D DevSecOps Repository and Pipeline Artefacts - GitLab

The complete remediated source code, CI/CD configuration, setup instructions, and DevSecOps pipeline artefacts for the WM240 security testing submission are available in the Warwick GitLab repository below.

Repository Access

- **Repository URL:**
`https://git-lab.cyber.warwick.ac.uk/ccse5653630/streeling-ims-devsecops-security`
- **Access Type:** Private Warwick GitLab repository
- **Branch Used for Submission:** `main`
- **CI/CD File:**
`.gitlab-ci.yml`
- **Pipeline Job Name:**
`devsecops_security_pipeline`

Repository Contents

- **Backend source code:** `backend/`
- **Frontend source code:** `frontend/`
- **Backend dependencies:** `backend/requirements.txt`
- **Frontend dependencies:** `frontend/package.json` and `frontend/package-lock.json`
- **Setup and verification guidance:** `README.md`
- **CI/CD security workflow:** `.gitlab-ci.yml`

Pipeline Artefacts

The GitLab CI/CD pipeline generates DevSecOps security evidence during execution, including Snyk reports, OWASP ZAP baseline scan reports, and application runtime logs.

- **Pipeline artifact archive:** `gitlab-devsecops-security-reports`
- **Snyk reports:** `snyk-backend.json` and `snyk-frontend.json`
- **OWASP ZAP reports:** `report_html.html`, `report_json.json`, and `report_md.md`
- **Runtime logs:** `backend-runtime.log` and `frontend-runtime.log`

E AI Declaration and Academic Integrity

Limited AI-assisted tools were used during the development and documentation of this coursework, in accordance with the module's AI Collaboration policy, alongside the University's wider AI usage policy.

ChatGPT was also used to assist with LaTeX formatting tasks, primarily including:

- Initial report layout and contents page generation (Figure 34)
- Harvard reference formatting through a separate `reference.bib` file (Figure 35)

Grammarly was used for:

- Minor grammar correction to improve readability
- Final proofreading

Comet was used to help identify:

- Relevant academic sources
- Relevant industry and blog sources (Figure 36)

All final written analysis, policy recommendations, and evaluative conclusions were independently completed and critically assessed by the author.

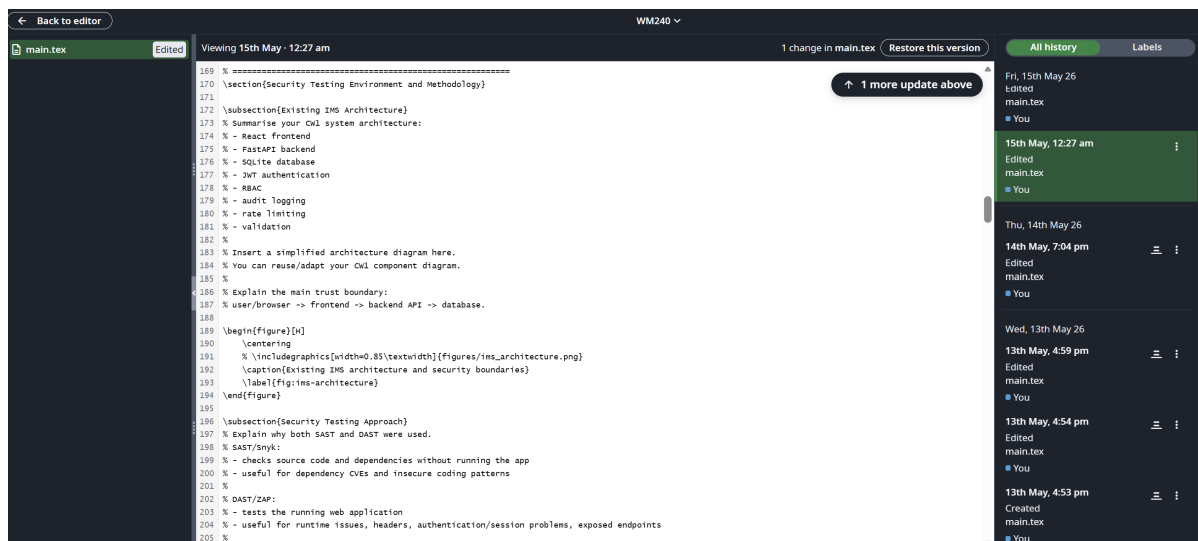


Figure 34: Example of AI-assisted report structuring and drafting guidance.

```
reference.bib
23
24 @article{aremu2025sastdast,
25   author = {Aremu, Oluwaferanmi and Liang, Warren},
26   title = {Static and Dynamic Application Security Testing (SAST and DAST) Integration
27           for Robust Secure Coding Practices},
28   year = {2025},
29   month = {July}
30 }
31 @misc{snyk2026whats,
32   author = {{Snyk}},
33   title = {What's Snyk?},
34   year = {2026},
35   url = {https://docs.snyk.io/discover-snyk/whats-snyk},
36   note = {Accessed: 2026-05-15}
37 }
38
39 @misc{pritesh2025owaspzap,
40   author = {Pritesh K},
41   title = {OWASP ZAP Tutorial: A Comprehensive Guide for DevSecOps},
42   year = {2025},
43   month = {May},
44   day = {23},
45   url = {https://devsecopsschool.com/blog/owasp-zap-tutorial-a-comprehensive-guide-for-
46           devsecops/},
47   note = {Accessed: 2026-05-15}
48 }
49 @misc{jayawardhana2025localhostserver,
50   author = {Jayawardhana, Shama},
51   title = {How to Test Your Web App Locally: 3 Beginner-Friendly Ways Using a Local HTTP
```

Figure 35: Use of a separate BibTeX file for Harvard reference management.

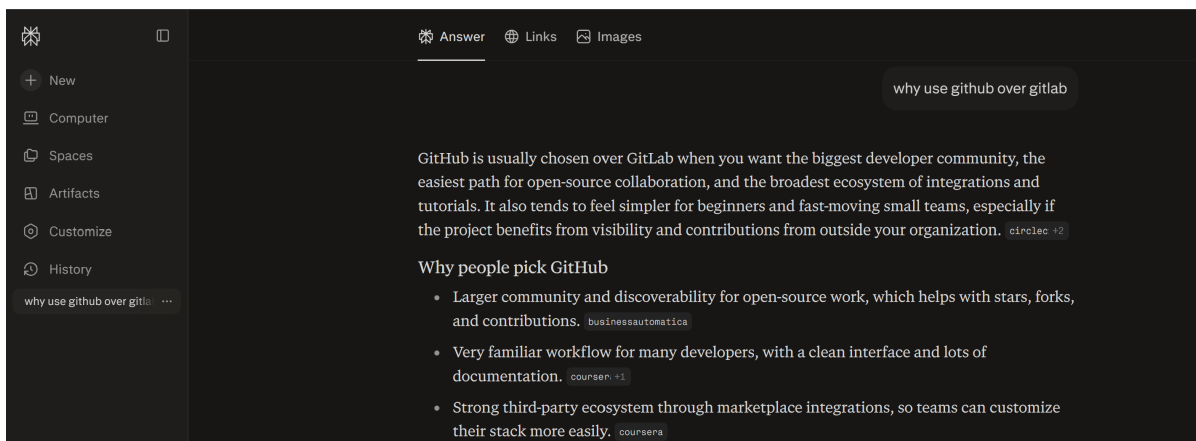


Figure 36: Example of AI-assisted source discovery and research support.